Software Architecture and Design Specification

Blood Bank Management System

Concise design based on SRS Version 1.0 and finalized project design choices

G S Retikka Prasad | PES2UG24AM057 | Section A

Jyothika M | PES2UG24AM070 | Section A
Gaganashree R | PES2UG24AM058 | Section A
Durugesh Bhergi | PES2UG24AM056 | Section A

Software Engineering UE24CS341A
Version 1.0 | 15 September 2026 Status
Draft

1.Introduction

1.1.Purpose

This document converts the Blood Bank Management System SRS into a concise implementation design. It defines the architecture, main modules, persistent entities, interfaces, security controls, and two central workflows for development and testing.

1.2.Scope

The design covers donor and recipient records, donations, blood inventory, blood requests, allocation and issue, authentication, role-based administration, reports, and audit logs. Clinical diagnosis, treatment decisions, and external hospital-system internals remain outside scope.

1.3.Audience

developers and testers, the course coordinator, project evaluators, and future maintainers.

1.4.Definitions

BBMS: Blood Bank Management System; RBAC: Role-Based Access Control; API: Application Programming Interface; RTM: Requirements Traceability Matrix; TLS: Transport Layer Security.

2.Document Overview

2.1.How to use this document

Sections 3 and 4 provide the architecture and detailed design. Requirement IDs refer to the supplied BBMS SRS v1.0 and are verified by the companion Software Test Plan.

2.2.Related Documents

- Blood Bank Management System Software Requirements Specification, Version 1.0.
- Blood Bank Management System Software Test Plan, Version 1.0.

3.Architecture

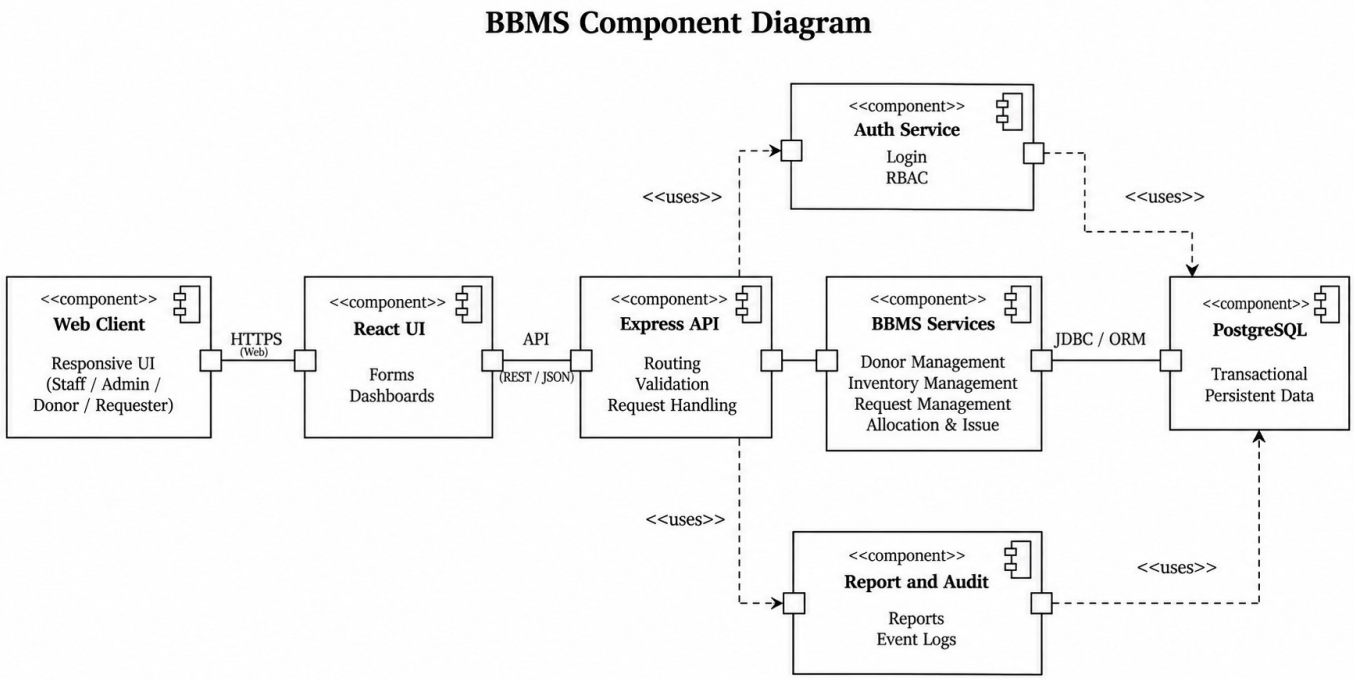
3.1.Goals and Constraints

Goals are correct inventory state, secure access, clear module boundaries, responsive common operations, and traceable administrative actions. Constraints include a student-project workload, browser-based access, relational storage, and project-defined donor eligibility and blood-compatibility rules that must be finalized before implementation.

3.2.Stakeholders and Concerns

Stakeholder	Primary Concerns
Blood-bank Staff	Fast data entry, accurate stock, safe request processing, clear errors.
Administrator	User and role management, reports, auditability, secure configuration.
Donor and Recipient	Accurate records, confidentiality, appropriate use of personal data.
Developer and Tester	Modularity, testability, consistent APIs, recoverable transactions.
Course Evaluator	SRS coverage, UML clarity, traceability, measurable acceptance criteria.

3.3.Component UML Diagram



3.4.Component Descriptions

- React UI - role-aware forms, dashboards, search, validation feedback, and reports.
- Express API - REST endpoints, request validation, authentication middleware, and error mapping.
- Auth Service - login, secure password verification, session control, and RBAC enforcement.
- BBMS Services - donor, donation, inventory, recipient, request, allocation, and issue rules.
- Report and Audit Service - summary queries and timestamped operational or administrative events.
- PostgreSQL - transactional records with keys, constraints, indexes, and controlled migrations.

3.5.Chosen Architecture Pattern and Rationale

A three-tier layered architecture separates presentation, application services, and data storage. The selected implementation uses React for the client, Node.js and Express for the server, and PostgreSQL for persistent data. It matches the project scale, keeps authorization and business rules in one backend, and supports independent module development. Microservices were rejected because they add deployment and consistency complexity without a project-scale benefit.

3.6.Technology Stack and Data Stores

Layer	Selected Technology	Purpose
Client	React, HTML, CSS	Responsive staff and admin interfaces.
Server	Node.js and Express	REST API, validation, workflows, RBAC.
Data	PostgreSQL	Transactional storage and reporting queries.
Security	TLS, secure cookies or tokens, Argon2 or bcrypt	Protected transport, sessions, and password hashing.
Quality	Jest, Supertest, Playwright	Unit, API, and end-to-end verification.

3.7.Risks and Mitigations

Risk	Mitigation
Eligibility or compatibility rules remain unclear	Approve rule tables before coding; keep them configurable and covered by tests.
Concurrent allocation of the same blood unit	Use database transactions, row locking, and status constraints.
Unauthorized access to sensitive records	Enforce backend RBAC, least privilege, TLS, secure sessions, and audit logs.
Incorrect expiry or inventory status	Centralize status logic and run boundary-date and reconciliation tests.
Data loss or partial workflow update	Use atomic transactions, backups, validation, and rollback on failure.

3.8.Traceability to Requirements

Requirement Group	Architecture Element
BBMS-F-001 to F-004	Donor Service, Donation records, donor UI.
BBMS-F-010 to F-014	Inventory Service, BloodUnit entity, inventory search and expiry logic.
BBMS-F-020 to F-027	Recipient, Request, Allocation and Issue services with transactional updates.
BBMS-F-030 to F-036	Auth Service, RBAC middleware, Admin UI, Reports and Audit Service.
BBMS-NF-001 to NF-007	Indexes, monitoring, responsive UI, modular layers, error-safe transactions.
BBMS-SR-001 to SR-005	TLS, password hashing, authorization, validation, parameterized queries, audit logs.

3.9.Security Architecture

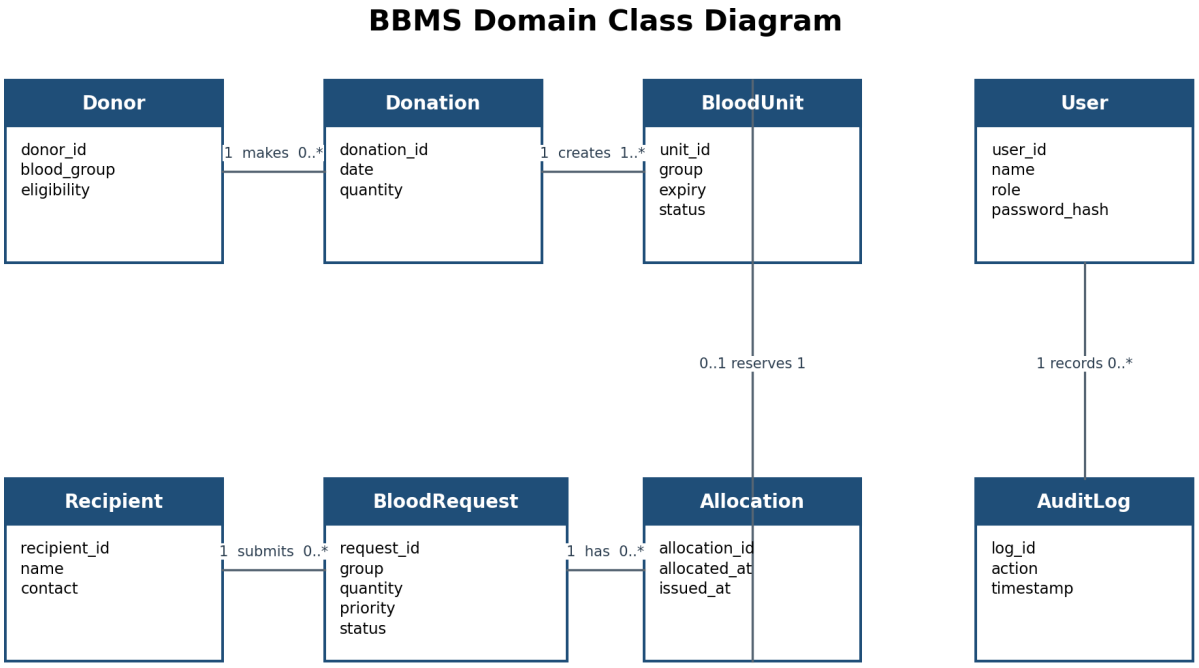
- Spoofing - secure password verification, session expiry, and login throttling.
- Tampering - server-side validation, parameterized queries, foreign keys, and transactional writes.
- Repudiation - audit user identity, action, record, timestamp, and outcome for significant events.
- Information disclosure - TLS, role-based data access, safe errors, and no plaintext passwords.
- Denial of service - request limits, bounded queries, timeouts, and health monitoring.
- Elevation of privilege - deny-by-default RBAC checked in the API, not only in the UI.

4.Design

4.1.Design Overview

Controllers validate transport-level input and call domain services. Services enforce eligibility, expiry, request-state, authorization, and allocation rules. Repositories isolate database operations. Allocation and issue operations execute as transactions so inventory and request states cannot diverge.

4.2.UML Class Diagram

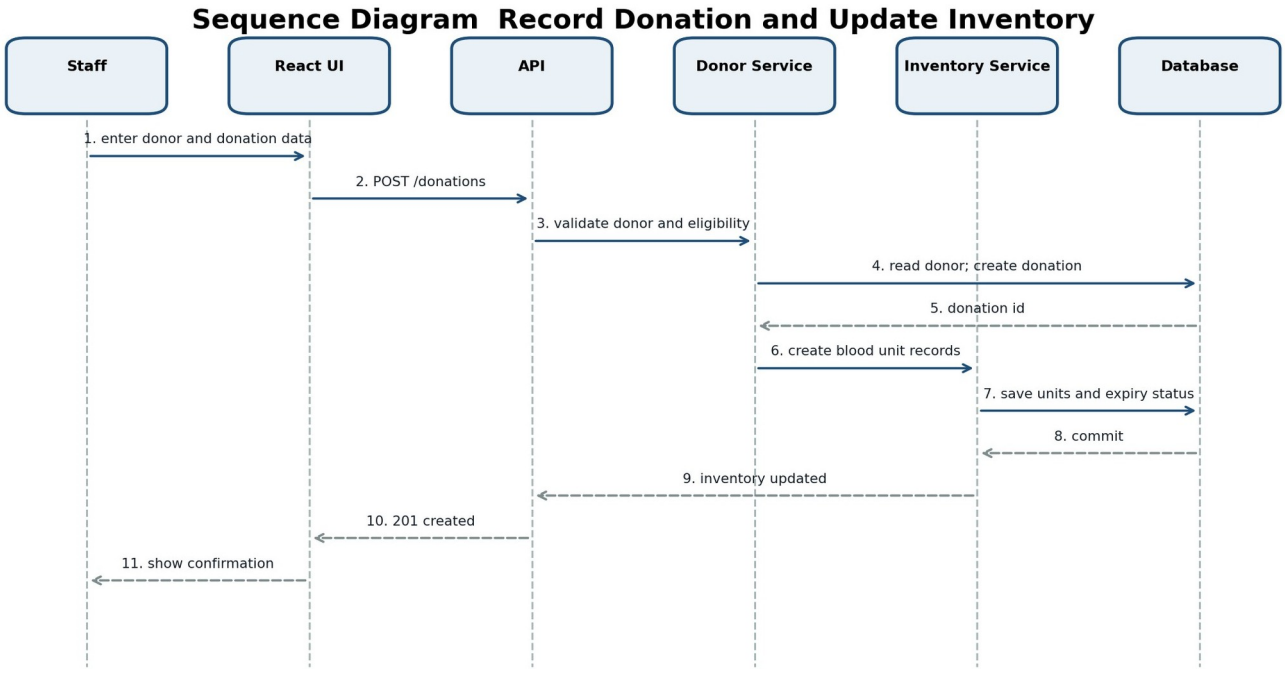


Key persistent entities and cardinalities

Blood units originate from recorded donations. Approved requests are satisfied through allocations, and an allocated unit can be issued at most once. Users generate audit entries for significant actions.

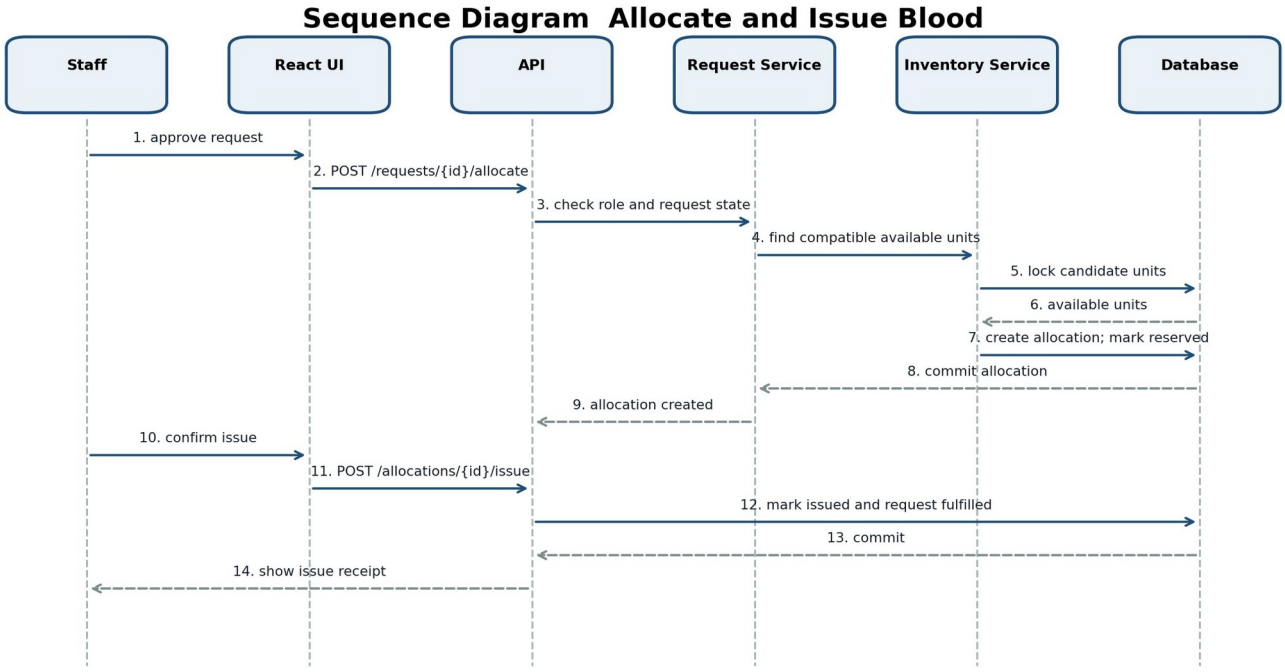
4.3.UML Sequence Diagrams

4.3.1.Record Donation and Update Inventory



The donation and its inventory units are committed together. A validation or database failure rolls back the complete operation.

4.3.2.Allocate and Issue Blood



The service checks authorization, request state, compatibility, availability, and expiry. Candidate units are locked during allocation to prevent double assignment.

4.4.API Design

Method and Endpoint	Purpose	Main Responses
POST /api/auth/login	Authenticate a registered user.	200 session; 401 invalid credentials; 429 throttled.
POST /api/donors	Create a donor record.	201 created; 400 validation; 409 duplicate.
POST /api/donations	Record donation and create stock units.	201 created; 400 ineligible or invalid; 404 donor.
GET /api/inventory	Filter stock by group, status, and expiry.	200 results; 400 invalid filter; 401 or 403 denied.
POST /api/requests	Create a recipient blood request.	201 created; 400 validation; 404 recipient.
POST /api/requests/{id}/allocate	Reserve compatible available units.	201 allocation; 409 insufficient or conflicting stock.
POST /api/allocations/{id}/issue	Record issue and fulfill the request.	200 issued; 409 invalid state; 403 denied.

4.5.Error Handling Logging and Monitoring

The API returns a stable error code, readable message, and correlation ID without stack traces or sensitive values. Logs record authentication failures, approvals, allocations, issues, user or role changes, and unexpected errors. Monitoring tracks response time, error rate, database health, failed logins, and inventory reconciliation exceptions.

4.6.UX Design

Role-based navigation shows only permitted tasks. Forms mark required fields, preserve entered values after validation errors, and require confirmation for approval, allocation, issue, and account changes.
Inventory status uses text labels in addition to color, and layouts remain usable on desktop and mobile browsers.

4.7.Open Issues and Next Steps

- Approve donor eligibility rules and the blood-group compatibility matrix.
- Confirm whether self-service accounts are in scope; the SRS currently defines Staff and Admin.
- Agree on the test-load size, backup retention, and deployment environment.

5.Appendices

Glossary: BBMS, RBAC, API, RTM, TLS. References: BBMS SRS v1.0, companion STP v1.0, and course SAD template. Diagram tools: Matplotlib-based UML renderings.